16.3 模型目录与配置

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~400,000 tokens，输出 ~32,000 tokens（本章合计）

OpenClaw 需要知道每个模型的能力参数——上下文窗口大小、是否支持推理、接受什么类型的输入。这些信息来自**模型目录**（Model Catalog）和**模型配置文件**（models.json），两者共同定义了系统可用的模型全景。

16.3.1 模型目录（`src/agents/model-catalog.ts`）

模型目录是系统在运行时发现的所有可用模型的列表。它通过 Pi Agent SDK 的 `ModelRegistry` 动态加载。

目录条目类型

```
// src/agents/model-catalog.ts
export type ModelCatalogEntry = {
  id: string;                        // 模型 ID（如 "claude-sonnet-4-20250514"）
  name: string;                      // 显示名称
  provider: string;                  // 提供者标识
  contextWindow?: number;            // 上下文窗口大小（Token 数）
  reasoning?: boolean;               // 是否支持推理/思考模式
  input?: Array<"text" | "image">;   // 支持的输入类型
};
```

目录加载

```
// src/agents/model-catalog.ts（简化版）
export async function loadModelCatalog(params?: {
  config?: OpenClawConfig;
  useCache?: boolean;
}): Promise<ModelCatalogEntry[]> {
  // 使用 Promise 缓存避免重复加载
  if (modelCatalogPromise && params?.useCache !== false) {
    return modelCatalogPromise;
  }

  modelCatalogPromise = (async () => {
    const cfg = params?.config ?? loadConfig();
    // 确保 models.json 存在
    await ensureOpenClawModelsJson(cfg);

    // 通过 Pi SDK 加载模型注册表
    const piSdk = await import("./pi-model-discovery.js");
    const agentDir = resolveOpenClawAgentDir();
    const authStorage = new piSdk.AuthStorage(join(agentDir, "auth.json"));
    const registry = new piSdk.ModelRegistry(authStorage, join(agentDir, "models.json")

    // 转换为 ModelCatalogEntry
    const entries = registry.getAll();
    return entries.map((entry) => ({
      id: entry.id,
      name: entry.name ?? entry.id,
      provider: entry.provider,
      contextWindow: entry.contextWindow,
      reasoning: entry.reasoning,
      input: entry.input,
    })).sort((a, b) => a.provider.localeCompare(b.provider) || a.name.localeCompare(b.n
  })();

  return modelCatalogPromise;
}
```

目录加载有几个重要特性：

1. **Promise 缓存**——使用模块级变量 `modelCatalogPromise` 缓存加载结果，整个进程生命周期内只加载一次（除非显式刷新）

2. **动态导入**——Pi SDK 通过 `import()` 动态加载，避免在不需要时引入大量依赖
3. **错误隔离**——动态导入放在 `try/catch` 内部，避免暂时性故障（如 `pnpm install` 期间 `node_modules` 被替换）导致缓存被一个 rejected Promise 永久污染

16.3.2 模型扫描：OpenRouter 免费模型探测

OpenClaw 支持通过 OpenRouter 接入数百个第三方模型。系统可以自动扫描 OpenRouter 上可用的免费模型，将它们添加到模型目录中。

模型扫描的流程：

1. 调用 OpenRouter 的模型列表 API
2. 过滤出免费模型（定价为 0）
3. 提取模型参数（上下文窗口、输入类型等）
4. 将结果合并到本地模型配置中

这使得用户可以无需手动配置就能使用 OpenRouter 上的免费模型，降低了入门门槛。

16.3.3 模型配置文件（`models.json`）与提供者配置

模型目录的数据来源之一是 Agent 目录下的 `models.json` 文件。这个文件定义了每个提供者的模型列表和连接参数。

配置合并策略

OpenClaw 的模型配置支持两种合并模式：

```
// src/agents/models-config.ts
const DEFAULT_MODE: NonNullable<ModelsConfig["mode"]> = "merge";
```

- **merge（默认）**——用户配置与内置提供者配置合并，用户可以添加新模型或覆盖已有模型的参数
- **replace**——用户配置完全替代内置配置

提供者配置合并

```
// src/agents/models-config.ts（简化版）
function mergeProviderModels(implicit: ProviderConfig, explicit: ProviderConfig): Provi
  const implicitModels = implicit.models ?? [];
  const explicitModels = explicit.models ?? [];

  // 用户配置的模型优先
  const seen = new Set(explicitModels.map((m) => m.id));
  const mergedModels = [
    ...explicitModels,
    ...implicitModels.filter((m) => !seen.has(m.id)),
  ];

  return { ...implicit, ...explicit, models: mergedModels };
}
```

合并规则是“用户优先”——如果用户和内置配置都定义了同一个模型 ID，使用用户的定义。这允许用户覆盖特定模型的参数（如自定义 base URL 或上下文窗口大小）。

隐式提供者

某些提供者不需要用户显式配置就可以使用。系统通过 `resolveImplicitProviders` 自动发现这些提供者：

```
// src/agents/models-config.providers.ts（概念）
export function resolveImplicitProviders(cfg: OpenClawConfig): Record<string, ProviderC
  const providers: Record<string, ProviderConfig> = {};

  // 如果检测到 AWS 凭据，自动添加 Bedrock 提供者
  const bedrock = resolveImplicitBedrockProvider(cfg);
  if (bedrock) providers["bedrock"] = bedrock;

  // 如果检测到 Copilot Token，自动添加 Copilot 提供者
  const copilot = resolveImplicitCopilotProvider(cfg);
  if (copilot) providers["github-copilot"] = copilot;

  return providers;
}
```

确保 models.json 存在

每次 Agent 启动时，系统确保 `models.json` 文件存在且是最新的：

```
// src/agents/models-config.ts（简化版）
export async function ensureOpenClawModelsJson(cfg?: OpenClawConfig, agentDir?: string)
  const resolved = agentDir ?? resolveOpenClawAgentDir();
  const filePath = path.join(resolved, "models.json");

  // 合并隐式提供者和用户配置
  const implicit = resolveImplicitProviders(cfg);
  const explicit = cfg?.models?.providers ?? {};
  const mode = cfg?.models?.mode ?? "merge";

  const providers = mode === "merge"
    ? mergeProviders({ implicit, explicit })
    : explicit;

  // 写入 models.json
  await fs.writeFile(filePath, JSON.stringify({ providers }, null, 2));
}
```

16.3.4 合成模型与特殊提供者

除了主流的 LLM 提供者（Anthropic、OpenAI、Google），OpenClaw 还支持多种特殊提供者和合成模型。

特殊提供者列表

提供者	API 类型	特点
Venice	OpenAI 兼容	隐私优先的 AI 推理
Chutes	OpenAI 兼容	开源模型托管
Z.AI	OpenAI 兼容	不支持 developer role
OpenRouter	OpenAI 兼容	数百个模型的统一入口
Bedrock	AWS SDK	通过 AWS 凭据认证
GitHub Copilot	专用协议	二阶 Token 认证

提供者	API 类型	特点
Qwen Portal	专用 API	阿里云通义千问

提供者 ID 标准化

如第 8.1 节所述， `normalizeProviderId` 处理了各种提供者名称的变体：

```
"z.ai" / "z-ai"      → "zai"
"opencode-zen"        → "opencode"
"qwen"                → "qwen-portal"
"kimi-code"           → "kimi-coding"
```

> **v2026.3.9 新增：OpenCode Go 提供者**
>
> v2026.3.9 新增了 `opencode-go` 提供者（ `src/commands/opencode-go-model-default.ts` ），与已有的 `opencode` （Zen）并列。两者在引导向导（Onboarding Wizard）中统一配置——共享同一个 API Key，但拆分为独立的运行时提供者。同时新增了 `MODELSTUDIO_API_KEY` 环境变量的 Shell-env 回退机制和隐式提供者自动发现功能。

合成模型

某些“模型”实际上是对其他模型的封装或组合。例如：

- **Claude CLI**——不是一个 API 模型，而是通过 Anthropic 的命令行工具执行，使用独立进程和 stdio 通信
- **Codex CLI**——OpenAI 的命令行编程助手，类似 Claude CLI 的运行模式

这些“CLI 模型”通过 `isCliProvider` 检测，走完全不同的执行路径（ `runCliAgent` 而非 `runEmbeddedPiAgent` ）。

本节小结

1. **模型目录**通过 Pi SDK 的 `ModelRegistry` 动态加载，包含每个模型的上下文窗口、推理能力和输入类型信息。目录使用 Promise 缓存避免重复加载。
2. **模型扫描**可以自动发现 OpenRouter 上的免费模型，降低用户的配置负担。
3. **models.json** 配置文件支持 merge 和 replace 两种合并模式，用户配置可以扩展或覆盖内置的提供者定义。隐式提供者（Bedrock、Copilot）可以自动发现。

4. **合成模型和特殊提供者**通过提供者 ID 标准化和 API 类型检测，被透明地集成到统一的模型选择和故障转移框架中。